

1. Описание условия задания

Цель работы: Сбалансировать дерево из букв вводимой строки (задача №6). Вывести его на экран в виде дерева. Составить хеш-таблицу, содержащую буквы и количество их вхождений во введенной строке. Вывести таблицу на экран. Реализовать операции добавления и удаления введенной буквы во всех структурах. Осуществить поиск введенной буквы в двоичном дереве поиска, в сбалансированном дереве и в хеш-таблице. Сравнить время поиска, объем памяти и количество сравнений при использовании различных структур данных.

2. Техническое задание

Внешняя спецификация программы:

Исходные данные:

- Исходная строка латинских символов.
- Выбор операции через консольное меню.

Выходные данные:

- Изображение дерева с подсветкой повторяющихся элементов или без.
- Изображение сбалансированного дерева с подсветкой повторяющихся элементов или без.
- Хэш-таблица, использующая цепочки.
- Хэш-таблица, использующая линейную адресацию.
- Сравнение времени выполнения и объема памяти операции поиска элементов в разных структурах данных.

Ограничения:

- Максимальный размер входной строки: 10 000 латинских символов.

Способ обращения: консольный ввод/вывод.

Команда запуска программы: ./app.exe. Дальнейшее взаимодействие осуществляется с помощью меню.

Аварийные ситуации:

- “ERROR_MEM” – Ошибка выделения памяти.
- “ERROR_IO” – Ошибка ввода.

Возможные действия, реализуемой программой:

ГЛАВНОЕ МЕНЮ	
BST-ДЕРЕВО	
01	- Добавить строку в BST-дерево.
02	- Удалить элемент из BST-дерева.
03	- Вывести BST-дерево.
04	- Поиск элемента в BST-дереве.
05	- Подсветить повторяющиеся элементы (BST).
06	- Удалить повторяющиеся элементы.
07	- Очистить BST-дерево.
08	- Сбалансировать BST-дерево (построить AVL).
AVL-ДЕРЕВО	
11	- Добавить строку в AVL-дерево.
12	- Удалить элемент из AVL-дерева.
13	- Вывод AVL-дерева.
14	- Поиск элемента в AVL-дереве.
15	- Подсветить повторяющиеся элементы (AVL).
16	- Очистка AVL-дерева.
ХЭШ-ТАБЛИЦА (ЦЕПОЧКИ)	
21	- Добавить строку в хеш-таблицу (цепочки).
22	- Удалить элемент из хеш-таблицы (цепочки).
23	- Поиск элемента в хеш-таблице (цепочки).
24	- Вывод хеш-таблицы (цепочки).
25	- Реструктуризация (цепочки).
26	- Очистка хеш-таблицы (цепочки).
ХЭШ-ТАБЛИЦА (ЛИНЕЙНАЯ АДРЕСАЦИЯ)	
31	- Добавить строку в хеш-таблицу (адресация).
32	- Удалить элемент из хеш-таблицы (адресация).
33	- Поиск элемента в хеш-таблице (адресация).
34	- Вывод хеш-таблицы (адресация).
35	- Реструктуризация (адресация).
36	- Очистка хеш-таблицы (адресация).
СИСТЕМНЫЕ ОПЕРАЦИИ	
100	- Замеры производительности (время, память, сравнения).
~~~~~	
0	- Выход из программы.

### 3. Внутренние структуры данных

В рамках выполнения данной лабораторной работы мной были реализованы следующие структуры данных:

Структура узла BST-дерева
---------------------------

<pre>typedef struct node_t {     char data;     int count;     struct node_t *left, *right; } bst_node_t;</pre>
-----------------------------------------------------------------------------------------------------------------

- **data** – хранящийся символ.
- **count** – количество вхождений символа.
- **left** – указатель на левого потомка.
- **right** – указатель на правого потомка.

Структура узла AVL-дерева
---------------------------

<pre>typedef struct avl_node_t {     char data;     int count;     int height;     struct avl_node_t *left, *right; } avl_node_t;</pre>
-----------------------------------------------------------------------------------------------------------------------------------------

- **data** – хранящийся символ.
- **count** – количество вхождений символа.
- **height** — высота поддерева с корнем в данном узле (используется для балансировки).
- **left** – указатель на левого потомка.
- **right** – указатель на правого потомка.

Структура узла хеш-таблицы с цепочками
----------------------------------------

<pre>typedef struct ht_chain_node {     char key;     int count;     struct ht_chain_node *next; } ht_chain_node_t;</pre>
---------------------------------------------------------------------------------------------------------------------------

- **key** — ключ (символ), по которому выполняется хеширование.
- **count** — количество вхождений символа.
- **next** — указатель на следующий узел в цепочке (при коллизии).

Структура хеш-таблицы с цепочками
-----------------------------------

```
typedef struct
{
    ht_chain_node_t **table;
    int size;
    int num_elements;
} hashtable_chain_t;
```

- **table** — массив указателей на головы цепочек (размер size).
- **size** — текущий размер таблицы (всегда простое число).
- **num_elements** — количество уникальных ключей, хранящихся в таблице.

Статус ячейки хеш-таблицы с открытой адресацией

```
typedef enum
{
    EMPTY,
    OCCUPIED,
    DELETED
} cell_status_t;
```

- **EMPTY** — ячейка никогда не использовалась.
- **OCCUPIED** — ячейка занята элементом.
- **DELETED** — ячейка содержала элемент, который был удалён (для корректного поиска).

Структура элемента хеш-таблицы с открытой адресацией

```
typedef struct
{
    char key;
    int count;
    cell_status_t status;
} ht_open_entry_t;
```

- **key** — СИМВОЛ-КЛЮЧ.
- **count** — количество вхождений.
- **status** — состояние ячейки (важно при поиске и вставке).

Структура хеш-таблицы с открытой адресацией

```
typedef struct
{
    ht_open_entry_t *table;
    int size;
    int num_elements;
} hashtable_open_t;
```

- **table** — массив ячеек фиксированного размера (size).
- **size** — размер таблицы (простое число, подобранное под данные).
- **num_elements** — количество активных (непустых и не удалённых) элементов.

## 4. Описание Алгоритмов

### Основной алгоритм программы:

1. Вывести меню;
2. Считать выбор пользователя и проверить его корректность;
3. Выполнить соответствующее действие;
4. Либо снова вывести меню с ожиданием выбора, либо завершить программу с сообщением об ошибке.

### Алгоритм балансировки BST-дерева (построение AVL-дерева):

1. Если BST-дерево (`bst_root`) пусто:
  - 1.1. Вывести сообщение, что балансировка невозможна.
  - 1.2. Завершить алгоритм.
2. Иначе:
  - 2.1. Извлечь все ключи из BST-дерева с учётом их количества (`extract_keys_with_duplicates`).
  - 2.2. Освободить память текущего AVL-дерева (`avl_free`).
  - 2.3. Инициализировать новый корень AVL-дерева (`avl_root = NULL`).
  - 2.4. Для каждого ключа из извлечённого массива:
    - 2.4.1. Вставить ключ в AVL-дерево (`avl_insert`).
    - 2.4.2. При неудачной вставке (нехватка памяти) завершить с ошибкой.
3. Вернуть управление пользователю (AVL-дерево построено на основе BST).

### Алгоритм реструктуризации хеш-таблицы с цепочками:

1. Если хеш-таблица (`ht`) не инициализирована:
  - 1.1. Завершить алгоритм.
2. Рассчитать новый размер таблицы (`new_size`):
  - 2.1. `new_size = ceil(текущее_число_элементов / 0.72)`.
  - 2.2. Найти ближайшее простое число, большее или равное `new_size`.
  - 2.3. Если полученное число не больше текущего размера, удвоить текущий размер и найти ближайшее простое.
3. Создать новую хеш-таблицу (`new_ht`) с размером `new_size`.
4. Перенести все элементы из старой таблицы (`ht`) в новую (`new_ht`):
  - 4.1. Для каждой ячейки старой таблицы:
    - 4.1.1. Для каждого узла в цепочке ячейки:
      - 4.1.1.1. Вставить ключ в новую таблицу (`ht_chain_insert`).
      - 4.1.1.2. Восстановить точное значение счётчика (`count`) в новой таблице.
5. Уничтожить старую хеш-таблицу (`ht_chain_destroy`).
6. Вернуть указатель на новую, реструктуризованную таблицу.

### **Алгоритм реструктуризации хеш-таблицы с линейной адресацией:**

1. Если хеш-таблица (`ht`) не инициализирована:
  - 1.1. Завершить алгоритм.
2. Рассчитать новый размер таблицы (`new\_size`):
  - 2.1.  $\text{new_size} = \text{ceil}(\text{текущее_число_элементов} * 1.2)$ .
  - 2.2. Найти ближайшее простое число, большее или равное `new\_size`.
  - 2.3. Если полученное число не больше текущего размера, удвоить текущий размер и найти ближайшее простое.
3. Создать новую хеш-таблицу (`new\_ht`) с размером `new\_size`.
4. Перенести все элементы из старой таблицы (`ht`) в новую (`new\_ht`):
  - 4.1. Для каждой ячейки старой таблицы:
    - 4.1.1. Если статус ячейки — `OCCUPIED`:
      - 4.1.1.1. Вставить ключ в новую таблицу (`ht\_open\_insert`).
      - 4.1.1.2. Восстановить точное значение счётчика (`count`) в новой таблице.
5. Уничтожить старую хеш-таблицу (`ht\_open\_destroy`).
6. Вернуть указатель на новую, реструктуризованную таблицу.

### **Алгоритм сравнения эффективности структур данных (замеры)**

1. Сгенерировать тестовую строку из 10000 случайных букв.
2. Построить структуры данных:
  - 2.1. BST-дерево — вставить все символы строки.
  - 2.2. AVL-дерево — вставить все символы строки.
  - 2.3. Хеш-таблица с цепочками — вставить все символы строки
  - 2.4. Хеш-таблица с линейной адресацией — вставить все символы строки.
3. Получить массив уникальных символов из строки.
4. Выполнить замеры (100000 итераций для усреднения):
  - 4.1. Для каждой структуры (BST, AVL, цепочки, адресация):
    - 4.1.1. Засечь время начала (`start = clock()`).
    - 4.1.2. Выполнить поиск каждого уникального символа в структуре, подсчитывая общее количество сравнений.
    - 4.1.3. Засечь время окончания (`end = clock()`).
    - 4.1.4. Накопить общее время и общее число сравнений.
5. Рассчитать средние значения:
  - 5.1. Среднее время поиска (мс) = (суммарное время / число итераций).
  - 5.2. Среднее количество сравнений = (суммарное число сравнений / (число итераций * число уникальных символов)).
6. Измерить объём занимаемой памяти для каждой структуры.
7. Вывести результаты в виде таблицы.

## Реализуемые функции:

// Функция очистки буфера ввода

1. void clear_input_buffer(void);

// Функция проверки, состоит ли строка только из латинских букв

2. int is_string(char *string);

// Функция вывода сообщения об ошибке по коду

3. void print_error(int rc);

// Функция вывода главного меню программы

4. void print_menu(void);

// Функция открытия изображения системной командой

5. void show_image(const char *image_path);

// Функция создания узла дерева

6. bst_node_t *bst_create_node(char data);

// Функция вставки символа в BST

7. bst_node_t *bst_insert(bst_node_t *root, char data);

// Функция поиска элемента в BST

8. bst_node_t *bst_search(bst_node_t *root, char key);

// Функция поиска с подсчётом сравнений

9. bst_node_t *bst_search_with_cmp(bst_node_t *root, char key, int *comparisons);

// Функция инфиксного обхода дерева

10. void in_order(bst_node_t *node, int depth);

// Функция префиксного обхода дерева

```
11. void pre_order(bst_node_t *node, int depth);

// Функция постфиксного обхода дерева
12. void post_order(bst_node_t *node, int depth);

// Функция удаления узла из BST
13. bst_node_t *bst_delete(bst_node_t *root, char key);

// Функция удаления дубликатов из BST
14. bst_node_t *bst_delete_duplicates(bst_node_t *root);

// Функция освобождения памяти дерева
15. void bst_free(bst_node_t *root);

// Функция вычисления высоты дерева
16. int bst_height(const bst_node_t *root);

// Функция расчёта объёма памяти дерева
17. size_t bst_memory_usage(const bst_node_t *root);

// Вспомогательная: подсчёт общего числа узлов
18. static int count_total_nodes_with_duplicates(const bst_node_t *node);

// Вспомогательная: заполнение массива ключей
19. static void fill_keys_array(const bst_node_t *node, char **keys_ptr);

// Функция извлечения всех ключей с учётом повторов
20. char *extract_keys_with_duplicates(const bst_node_t *root, int *total_count);

// Вспомогательная: генерация DOT-кода с подсветкой
21. void generate_dot_highlight(bst_node_t *root, FILE *file, int highlight_duplicates);
```

// Функция создания DOT-файла дерева

22. int bst_create_dot(bst_node_t *root, const char *filename, int highlight_duplicates);

// Функция получения высоты узла AVL

23. int avl_height(const avl_node_t *node);

// Функция вычисления баланса узла

24. int avl_get_balance(const avl_node_t *node);

// Функция создания узла AVL-дерева

25. avl_node_t *avl_create_node(char data);

// Функция правого поворота узла

26. avl_node_t *avl_rotate_right(avl_node_t *y);

// Функция левого поворота узла

27. avl_node_t *avl_rotate_left(avl_node_t *x);

// Функция вставки в AVL-дерево

28. avl_node_t *avl_insert(avl_node_t *root, char data);

// Функция удаления из AVL-дерева

29. avl_node_t *avl_delete(avl_node_t *root, char key);

// Функция поиска в AVL-дереве

30. avl_node_t *avl_search(avl_node_t *root, char key);

// Функция поиска с подсчётом сравнений

31. avl_node_t *avl_search_with_cmp(avl_node_t *root, char key, int *comparisons);

// Функция освобождения памяти AVL-дерева

32. void avl_free(avl_node_t *root);

// Функция расчёта объёма памяти AVL-дерева

33. size_t avl_memory_usage(const avl_node_t *root);

// Функция получения высоты AVL-дерева

34. int avl_tree_height(const avl_node_t *root);

// Вспомогательная: генерация DOT-кода с подсветкой

35. static void avl_generate_dot_highlight(avl_node_t *root, FILE *file, int highlight_duplicates);

// Функция создания DOT-файла AVL-дерева

36. int avl_create_dot(avl_node_t *root, const char *filename, int highlight_duplicates);

// Функция вывода дерева в файл

37. void avl_print_to_file(avl_node_t *root, FILE *file, int depth);

// Вспомогательная: проверка простого числа

38. static int is_prime(int n);

// Вспомогательная: нахождение следующего простого

39. static int next_prime(int n);

// Вспомогательная: хеш-функция

40. static unsigned int hash_func(char key, int size);

// Функция создания хеш-таблицы

41. hashtable_open_t *ht_open_create(int initial_size);

// Функция уничтожения хеш-таблицы

42. void ht_open_destroy(hashtable_open_t *ht);

// Функция вставки в хеш-таблицу

```
43. int ht_open_insert(hashtable_open_t *ht, char key);
```

// Функция поиска в хеш-таблице

```
44. int ht_open_search(hashtable_open_t *ht, char key, int *comparisons);
```

// Функция удаления из хеш-таблицы

```
45. int ht_open_delete(hashtable_open_t *ht, char key);
```

// Функция вывода хеш-таблицы

```
46. void ht_open_print(hashtable_open_t *ht);
```

// Функция реструктуризации таблицы

```
47. hashtable_open_t *ht_open_rehash(hashtable_open_t *ht);
```

// Функция расчёта среднего числа сравнений

```
48. double ht_open_avg_comparisons(hashtable_open_t *ht);
```

// Вспомогательная: хеш-функция

```
49. static unsigned int hash_func(char key, int size);
```

// Функция создания хеш-таблицы

```
50. hashtable_chain_t *ht_chain_create(int initial_size);
```

// Функция уничтожения хеш-таблицы

```
51. void ht_chain_destroy(hashtable_chain_t *ht);
```

// Функция вставки в хеш-таблицу

```
52. int ht_chain_insert(hashtable_chain_t *ht, char key);
```

// Функция поиска в хеш-таблице

```
53. int ht_chain_search(hashtable_chain_t *ht, char key, int *comparisons);
```

// Функция удаления из хеш-таблицы

54. int ht_chain_delete(hashtable_chain_t *ht, char key);

// Функция вывода хеш-таблицы

55. void ht_chain_print(hashtable_chain_t *ht);

// Функция реструктуризации таблицы

56. hashtable_chain_t *ht_chain_rehash(hashtable_chain_t *ht);

// Функция расчёта среднего числа сравнений

57. double ht_chain_avg_comparisons(hashtable_chain_t *ht);

// Вспомогательная: генерация случайной строки

58. static void generate_random_string(char *buf, int size);

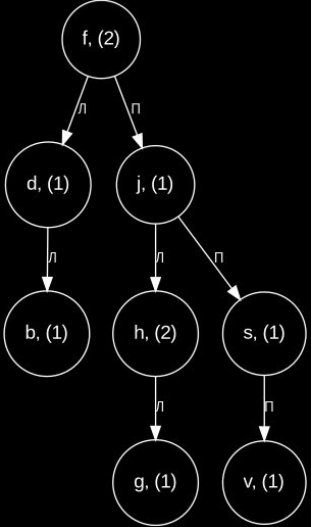
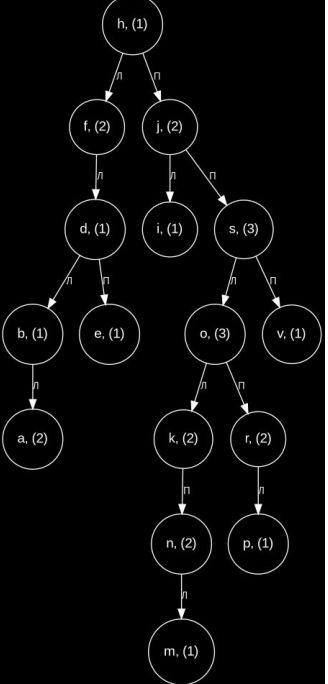
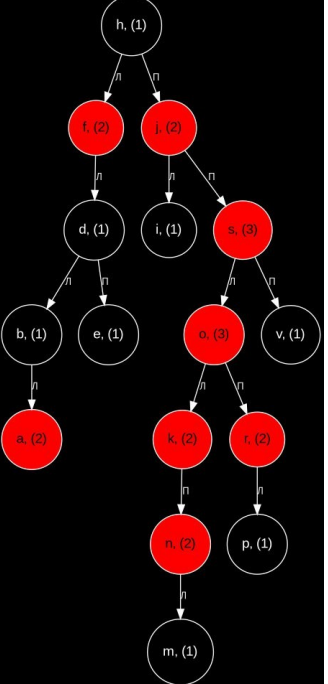
// Вспомогательная: получение уникальных символов

59. static char *get_unique_chars(const char *str, int *n_unique);

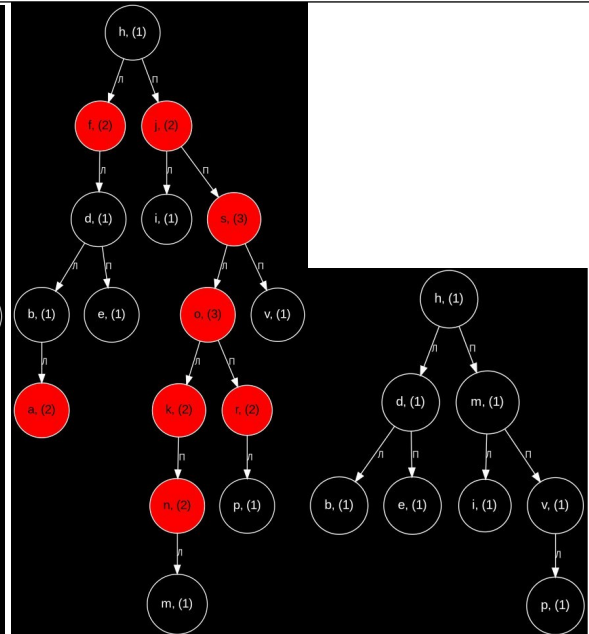
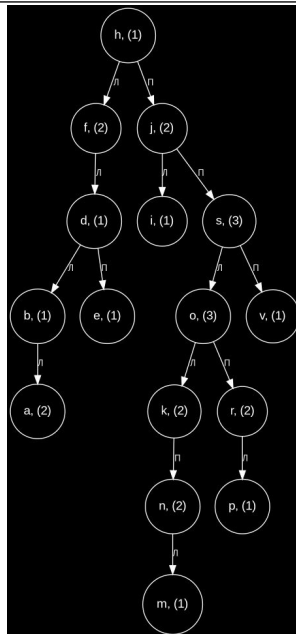
// Функция сравнения производительности структур

60. void compare_all_structures(void);

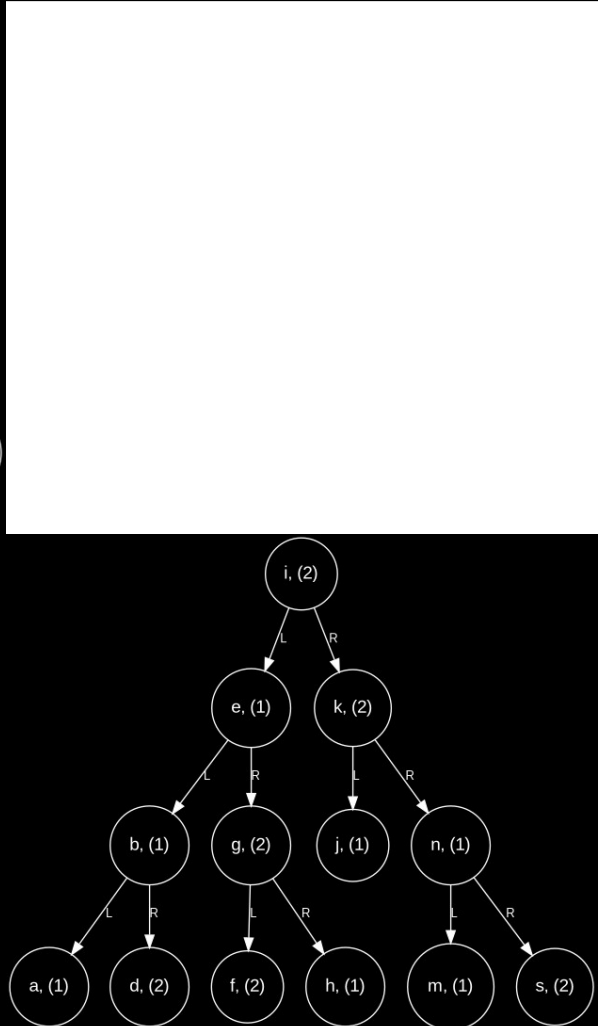
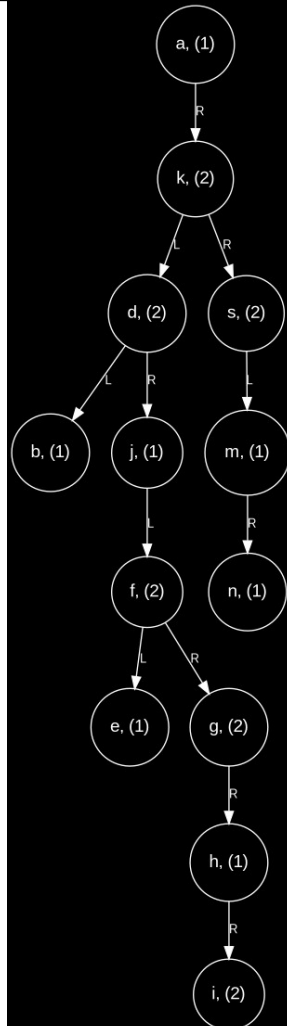
## 5. Тесты

## Позитивные тесты:	
Действие	Результат
Вывод пустого дерева	“Дерево пустое”
Вывод дерева из одного узла	
Вывод дерева из нескольких узлов: Строка: “fjdbsfhvgh”	
Вывод дубликатов	     

## Удаление дубликатов



## Балансировка дерева



Вывод реструктуризации хэш-таблицы цепочек	Хеш-таблица (цепочки), размер = 11, элементов = 21: <table> <tr><th>Инд.</th><th>Содержимое</th></tr> <tr><td>0</td><td>---</td></tr> <tr><td>1</td><td>---</td></tr> <tr><td>2</td><td>---</td></tr> <tr><td>3</td><td>---</td></tr> <tr><td>4</td><td>'B'(1)</td></tr> <tr><td>5</td><td>'b'(7)</td></tr> <tr><td>6</td><td>'D'(2) → 'c'(1)</td></tr> <tr><td>7</td><td>'d'(8)</td></tr> <tr><td>8</td><td>'F'(1)</td></tr> <tr><td>9</td><td>'G'(3) → 'f'(8)</td></tr> <tr><td>10</td><td>'H'(3) → 'g'(3)</td></tr> <tr><td>11</td><td>'h'(7)</td></tr> <tr><td>12</td><td>'J'(2) → 'i'(3)</td></tr> <tr><td>13</td><td>'j'(3)</td></tr> <tr><td>14</td><td>'k'(5)</td></tr> <tr><td>15</td><td>---</td></tr> <tr><td>16</td><td>---</td></tr> <tr><td>17</td><td>---</td></tr> <tr><td>18</td><td>---</td></tr> <tr><td>19</td><td>---</td></tr> <tr><td>20</td><td>---</td></tr> <tr><td>21</td><td>'S'(1)</td></tr> <tr><td>22</td><td>'s'(7)</td></tr> <tr><td>23</td><td>'U'(1)</td></tr> <tr><td>24</td><td>'V'(2) → 'u'(3)</td></tr> <tr><td>25</td><td>'v'(5)</td></tr> <tr><td>26</td><td>---</td></tr> <tr><td>27</td><td>---</td></tr> <tr><td>28</td><td>---</td></tr> <tr><td>29</td><td>---</td></tr> <tr><td>30</td><td>---</td></tr> </table>     Хеш-таблица (цепочки), размер = 11, элементов = 21: <table> <tr><th>Инд.</th><th>Содержимое</th></tr> <tr><td>0</td><td>'B'(1) → 'c'(1)</td></tr> <tr><td>1</td><td>'d'(8)</td></tr> <tr><td>2</td><td>'D'(2)</td></tr> <tr><td>3</td><td>'f'(8)</td></tr> <tr><td>4</td><td>'F'(1) → 'g'(3)</td></tr> <tr><td>5</td><td>'G'(3) → 'h'(7) → 's'(7)</td></tr> <tr><td>6</td><td>'S'(1) → 'H'(3) → 'i'(3)</td></tr> <tr><td>7</td><td>'u'(3) → 'j'(3)</td></tr> <tr><td>8</td><td>'J'(2) → 'U'(1) → 'v'(5) → 'k'(5)</td></tr> <tr><td>9</td><td>'V'(2)</td></tr> <tr><td>10</td><td>'b'(7)</td></tr> </table>	Инд.	Содержимое	0	---	1	---	2	---	3	---	4	'B'(1)	5	'b'(7)	6	'D'(2) → 'c'(1)	7	'd'(8)	8	'F'(1)	9	'G'(3) → 'f'(8)	10	'H'(3) → 'g'(3)	11	'h'(7)	12	'J'(2) → 'i'(3)	13	'j'(3)	14	'k'(5)	15	---	16	---	17	---	18	---	19	---	20	---	21	'S'(1)	22	's'(7)	23	'U'(1)	24	'V'(2) → 'u'(3)	25	'v'(5)	26	---	27	---	28	---	29	---	30	---	Инд.	Содержимое	0	'B'(1) → 'c'(1)	1	'd'(8)	2	'D'(2)	3	'f'(8)	4	'F'(1) → 'g'(3)	5	'G'(3) → 'h'(7) → 's'(7)	6	'S'(1) → 'H'(3) → 'i'(3)	7	'u'(3) → 'j'(3)	8	'J'(2) → 'U'(1) → 'v'(5) → 'k'(5)	9	'V'(2)	10	'b'(7)
Инд.	Содержимое																																																																																								
0	---																																																																																								
1	---																																																																																								
2	---																																																																																								
3	---																																																																																								
4	'B'(1)																																																																																								
5	'b'(7)																																																																																								
6	'D'(2) → 'c'(1)																																																																																								
7	'd'(8)																																																																																								
8	'F'(1)																																																																																								
9	'G'(3) → 'f'(8)																																																																																								
10	'H'(3) → 'g'(3)																																																																																								
11	'h'(7)																																																																																								
12	'J'(2) → 'i'(3)																																																																																								
13	'j'(3)																																																																																								
14	'k'(5)																																																																																								
15	---																																																																																								
16	---																																																																																								
17	---																																																																																								
18	---																																																																																								
19	---																																																																																								
20	---																																																																																								
21	'S'(1)																																																																																								
22	's'(7)																																																																																								
23	'U'(1)																																																																																								
24	'V'(2) → 'u'(3)																																																																																								
25	'v'(5)																																																																																								
26	---																																																																																								
27	---																																																																																								
28	---																																																																																								
29	---																																																																																								
30	---																																																																																								
Инд.	Содержимое																																																																																								
0	'B'(1) → 'c'(1)																																																																																								
1	'd'(8)																																																																																								
2	'D'(2)																																																																																								
3	'f'(8)																																																																																								
4	'F'(1) → 'g'(3)																																																																																								
5	'G'(3) → 'h'(7) → 's'(7)																																																																																								
6	'S'(1) → 'H'(3) → 'i'(3)																																																																																								
7	'u'(3) → 'j'(3)																																																																																								
8	'J'(2) → 'U'(1) → 'v'(5) → 'k'(5)																																																																																								
9	'V'(2)																																																																																								
10	'b'(7)																																																																																								
Вывод и реструктуризации хэш-таблицы линейной адресации	Хеш-таблица (линейная адресация), размер = 11, элементов = 8: <table> <tr><th>Инд.</th><th>Содержимое</th></tr> <tr><td>0</td><td>'c'( 1)</td></tr> <tr><td>1</td><td>'d'( 5)</td></tr> <tr><td>2</td><td>'e'( 1)</td></tr> <tr><td>3</td><td>'f'( 2)</td></tr> <tr><td>4</td><td>'g'( 2)</td></tr> <tr><td>5</td><td>'s'( 4)</td></tr> <tr><td>6</td><td>---</td></tr> <tr><td>7</td><td>---</td></tr> <tr><td>8</td><td>'v'( 2)</td></tr> <tr><td>9</td><td>---</td></tr> <tr><td>10</td><td>'b'( 1)</td></tr> </table>     Хеш-таблица (линейная адресация), размер = 17, элементов = 11: <table> <tr><th>Инд.</th><th>Содержимое</th></tr> <tr><td>0</td><td>'f'( 5)</td></tr> <tr><td>1</td><td>'g'( 5)</td></tr> <tr><td>2</td><td>'t'( 1)</td></tr> <tr><td>3</td><td>'v'( 2)</td></tr> <tr><td>4</td><td>'w'( 2)</td></tr> <tr><td>5</td><td>'b'( 2)</td></tr> <tr><td>6</td><td>---</td></tr> <tr><td>7</td><td>---</td></tr> <tr><td>8</td><td>---</td></tr> <tr><td>9</td><td>---</td></tr> <tr><td>10</td><td>---</td></tr> <tr><td>11</td><td>---</td></tr> <tr><td>12</td><td>'r'( 1)</td></tr> <tr><td>13</td><td>'s'( 7)</td></tr> <tr><td>14</td><td>'c'( 1)</td></tr> <tr><td>15</td><td>'d'( 6)</td></tr> <tr><td>16</td><td>'e'( 4)</td></tr> </table>	Инд.	Содержимое	0	'c'( 1)	1	'd'( 5)	2	'e'( 1)	3	'f'( 2)	4	'g'( 2)	5	's'( 4)	6	---	7	---	8	'v'( 2)	9	---	10	'b'( 1)	Инд.	Содержимое	0	'f'( 5)	1	'g'( 5)	2	't'( 1)	3	'v'( 2)	4	'w'( 2)	5	'b'( 2)	6	---	7	---	8	---	9	---	10	---	11	---	12	'r'( 1)	13	's'( 7)	14	'c'( 1)	15	'd'( 6)	16	'e'( 4)																												
Инд.	Содержимое																																																																																								
0	'c'( 1)																																																																																								
1	'd'( 5)																																																																																								
2	'e'( 1)																																																																																								
3	'f'( 2)																																																																																								
4	'g'( 2)																																																																																								
5	's'( 4)																																																																																								
6	---																																																																																								
7	---																																																																																								
8	'v'( 2)																																																																																								
9	---																																																																																								
10	'b'( 1)																																																																																								
Инд.	Содержимое																																																																																								
0	'f'( 5)																																																																																								
1	'g'( 5)																																																																																								
2	't'( 1)																																																																																								
3	'v'( 2)																																																																																								
4	'w'( 2)																																																																																								
5	'b'( 2)																																																																																								
6	---																																																																																								
7	---																																																																																								
8	---																																																																																								
9	---																																																																																								
10	---																																																																																								
11	---																																																																																								
12	'r'( 1)																																																																																								
13	's'( 7)																																																																																								
14	'c'( 1)																																																																																								
15	'd'( 6)																																																																																								
16	'e'( 4)																																																																																								
Поиск существующего и несуществующего элементов в дереве.																																																																																									
Сравнение времени и затраченной памяти.																																																																																									

## ## Негативные тесты:

1. Ввод невалидного значения при выборе меню.
2. Ввод не латинского символа при вводе строки.

## 6. Оценка эффективности

Количество запусков – 100 000 раз. Входная строка – 10 000 символов.

Сравнение алгоритма поиска элементов

Структура данных	Время (мкс)	Память (байт)	Всего сравнений	Ср. сравнений
BST	1.555	1248	345	6.635
AVL	1.162	1664	261	5.019
Хеш (цепочки)	0.416	1416	52	1.000
Хеш (адресация)	0.558	804	52	1.000

Наблюдения результатов по эффективности:

1. Хеш-таблицы значительно превосходят деревья по скорости поиска.
  - 1.1. При всех размерах данных хеш-таблицы демонстрируют время поиска в 2–4 раза меньше, чем даже сбалансированное AVL-дерево.
  - 1.2. Например, при  $N=10\,000$ : время поиска в хеш-таблице (цепочки) — 0.416 мкс, в AVL — 1.162 мкс, в BST — 1.555 мкс.
  - 1.3. Это подтверждает теоретическую сложность  $O(1)$  для хеш-таблиц против  $O(\log N)$  для сбалансированных деревьев.
2. Сбалансированное AVL-дерево эффективнее несбалансированного BST.
  - 2.1. AVL-дерево показывает на 25% меньше сравнений и на 25% меньше время поиска по сравнению с BST.
  - 2.2. Это связано с гарантированной сбалансированностью: высота AVL-дерева близка к  $1.44 \cdot \log_2(N)$ , в то время как BST может вырождаться в список (хотя в данном тесте использовалась случайная строка, поэтому вырождения не произошло).
3. Память: хеш-таблицы и деревья имеют сопоставимый расход памяти.
  - 3.1. AVL-дерево занимает 1664 байта — наибольший объём из-за хранения поля height.
  - 3.2. Хеш-таблица с линейной адресацией использует наименьший объём (804 байта), так как не требует указателей и хранит данные компактно в массиве.
  - 3.3. Цепочки потребляют больше памяти (1416 байт), чем адресация, из-за накладных расходов на next-указатели.
4. Среднее число сравнений подтверждает теоретические оценки.
  - 4.1. В хеш-таблицах среднее число сравнений равно 1.0, что означает отсутствие коллизий или их минимальное количество — результат эффективной хеш-функции и подходящего размера таблицы.

5. Выбор хеш-функции и размера таблицы критичен.
  - 5.1. Использована простая и эффективная хеш-функция  $h(c) = (\text{unsigned char})c \% m$ , обеспечивающая равномерное распределение символов.
  - 5.2. Размер таблицы подобран осмысленно: для цепочек —  $m \approx n / 0.72$ , для адресации —  $m \approx n * 1.2$ , и в обоих случаях округлѐн до ближайшего простого числа. Это минимизирует коллизии и обеспечивает высокую производительность.
6. Реструктуризация позволяет поддерживать эффективность при росте данных.
  - 6.1. Если бы таблицы были созданы с фиксированным малым размером (например, 11), среднее число сравнений выросло бы до 4–10, а время поиска — в 5–10 раз.
  - 6.2. Принцип реструктуризации — автоматическое увеличение размера таблицы и перехеширование при превышении целевого коэффициента заполнения (0.72 для цепочек, 0.7 для адресации) — гарантирует сохранение сложности  $O(1)$ .

## 7. Выводы

Хеш-таблицы демонстрируют наивысшую эффективность по времени благодаря  $O(1)$  сложности поиска, при этом потребляя сопоставимый или даже меньший объём памяти, чем деревья. Их производительность напрямую зависит от качества хеш-функции и правильного выбора размера таблицы.

AVL-дерево обеспечивает стабильную производительность с  $O(\log N)$  сложностью и предсказуемой глубиной, что делает его надёжным выбором, если хеш-функция неприменима или данные часто изменяются в порядке, вызывающем коллизии.

BST в несбалансированном виде уступает AVL и не рекомендуется для задач с высокими требованиями к производительности.

Таким образом, хеш-таблицы целесообразно использовать в задачах, где критична максимальная скорость поиска, а AVL-деревья — когда требуется гарантированная логарифмическая сложность и поддержка упорядоченного обхода. Выбор конкретной реализации хеш-таблицы (цепочки vs адресация) зависит от компромисса между памятью и сложностью реализации удаления.

## 8. Ответы на контрольные вопросы

### 1. Чем отличается идеально сбалансированное дерево от AVL дерева?

Идеально сбалансированное дерево — левое и правое поддеревья каждого узла содержат ровно равное или отличающееся на 1 число узлов. AVL-дерево требует, чтобы высоты поддеревьев отличались не более чем на 1. AVL — менее строгое условие, но проще поддерживать.

### 2. Чем отличается поиск в AVL-дереве от поиска в дереве двоичного поиска?

Алгоритм поиска одинаков (рекурсивный спуск по ключу), но в AVL-дереве глубина гарантированно  $O(\log n)$ , а в BST может вырождаться в  $O(n)$ .

### 3. Что такое хеш-таблица, каков принцип ее построения?

Хеш-таблица — структура данных, использующая массив и хеш-функцию (ключ  $\rightarrow$  индекс), для обеспечения среднего времени поиска  $O(1)$ .

### 4. Что такое коллизии? Каковы методы их устранения.

Коллизии — разные ключи дают одинаковый хеш-индекс. Методы устранения:

Открытое хеширование (цепочки),

Закрытое хеширование (линейная/квадратичная адресация, двойное хеширование).

### 5. В каком случае поиск в хеш-таблицах становится неэффективен?

Поиск неэффективен при высокой загрузке таблицы ( $>70\%$  для адресации), плохой хеш-функции (много коллизий) или длинных цепочках (для цепочек) — время растёт до  $O(n)$ .

### 6. Эффективность поиска в AVL деревьях, в дереве двоичного поиска, в хеш-таблицах и в файле.

Эффективность поиска:

Файл (линейный):  $O(n)$

BST (в среднем):  $O(\log n)$ , в худшем —  $O(n)$

AVL:  $O(\log n)$  (гарантированно)

Хеш-таблица (в среднем):  $O(1)$ , в худшем —  $O(n)$